1. Unfortunately, no picture since I don't want to get sued by Meta, but I architected and coded a graph-based platform driver framework. Our bringup platform was very, very, very flexible, on top of the usual complexity of building a server-class motherboard. For example, to talk JTAG from the onboard FTDI to (let's call it) Part A, a JTAG level shifter needed to be taken out of reset. The reset line was controlled via an I2C-based GPIO expander, and to get to the GPIO expander, we needed to toggle an I2C mux. To even write out I2C to the bus, we had to change the bus into I2C mode by driving a GPIO from the FTDI, and then we needed to use the same pins we just used for the FTDI GPIO to write I2C traffic out, which required reconfiguring the FTDI. You can see how this can get complicated when there are dozens of devices sharing various muxes, buses (with different protocols), and reset lines. With a computer-readable model of this connectivity, the system could:

   - automatically perform these tasks with a simple interface, e.g., system.go_do_this_jtag_thing_please()
   - support parallelism to manage resource contention on shared buses and reset lines—e.g., prevent talking I2C to Device B via Bus C because Bus C is currently being used by Device A.
   - provide many, many other quality-of-life features like optional safeguards, dynamically working around broken devices, removing redundant operations, automated platform testing, etc.

2. I use basic vim with the following ~/.vimrc:

   ```
   :set expandtab
   :set tabstop=4
   :set shiftwidth=4
   ```

   I used to use various other editors/IDEs, but about 5 years ago, I switched over to Vim. When connecting to a lab machine or an embedded system, they frequently only have Vim available, so it would always be a hiccup to use Vim rather than my standard setup. I eventually became tired of this and decided to just learn to use vim well so I could comfortably and efficiently work on any system. It also has the bonus benefit of minimizing the amount of time I spend faffing about with fancy text editors.

3. First, I would like to narrow down whether the part is actually misbehaving or if it's just an issue with the communication channel. We could put a scope on the part to ensure communications are getting to it and we are getting nothing back. If we do see garbage, or if another communication channel to the part works, we would probably need to debug the specific device rather than the platform (unless we are otherwise suspicious of power delivery or reset sequencing).

   Assuming it is truly not responding, I would check the reset and power lines. If one is not at the expected state, we may be able to trace it back through the platform to find out what is wrong; for example, if a power line is low, we can trace back to the power stage to ensure its resets and input power are appropriate, and so on. Hopefully, this would either lead us back to a device with firmware or a specific circuit that is malfunctioning. At this point, we should hopefully have an on-board signal that is changing unexpectedly, and we can use it as a trigger on a scope/logic analyzer with the other channels hooked up to potentially interesting signals. While this will be much slower, we can then capture the transient state as the fault occurs to help root-cause the device/circuit. While waiting for the fault to occur, I would be messing around with the system trying to reproduce it faster: increasing load, decreasing load, trying to exercise every path on the board, etc. With the scope captures, we should be able to continue the same game as before of checking signals for unexpected behavior, then checking the signals that influence that one, etc.

   For a real incident in a similar vein, we had an I3C bus from a server SoC to a DIMM that would, once every 100 transactions, get an unexpected NACK and hang the bus. Since we had other priorities at the time, we just worked around this for a few days, and in that time, noticed different boards failed at different rates. Since the exact same sequence was intermittently working, I first lowered the bus speed, and the problem went away entirely. I then rolled out a quick config change to lower everyone's bus speed so other people would not trip on the problem (albeit at a slower test speed) and went back to debugging.

   I hooked up a scope to the bus to see exactly how it was failing. Unfortunately, this caused it to no longer fail. I then went around and dangled a jumper wire off of all the platforms and increased the bus speed back to the target, allowing folks to resume running at max test speed. Since lowering the speed and adding extra line capacitance both fixed the problem, it implied that there were some noise issues that were magnified by manufacturing variances. Since the rise time looked to have some margin on the scope, I raised the impedance of the I3C driver and it resolved the issue. I was able to verify on a scope that we were still getting rise times

and voltage levels within spec.

4. One recurring grievance I have is when engineers over-optimize a code flow in the pursuit of saving a few instructions, even when the performance gain is not required or beneficial. This leads to code that is more likely to be buggy and harder to maintain in the future. For example, at my previous role, server system boot firmware runs once, and then the system runs for months without ever restarting again. I was frequently pushing back on complicated optimizations that sacrificed readability and simplicity. I think these two things are key to being able to make quick bug fixes and improvements in the future, without inadvertently introducing more problems. Based on my experience, this does seem to pay off, as when crunch time comes, we spend less time fighting code and more time fixing bugs or adding features. I think it also makes it way easier to do quick fixes or hacks without accidentally violating expectations for some performance optimization.

5. Halfway through validation of our first test chip, the validation team decided they wanted to run a specific type of pattern that is designed to be run during silicon manufacturing (IEEE 1450 STIL) that we were not set up to run in the lab. As we initially planned not to run these patterns, there was no infrastructure for it, and writing a proper language parser would take longer than our timeline allowed (plus, I don't know how to write a language parser...).

   I took a quick look at the patterns we wanted to run, and they all used fairly basic language features: loops, macros, functions, etc. I then spent an afternoon constructing a monstrous series of regex replaces (over 200 characters) that transpiled the STIL file to Python, which we could then run through our existing Python infrastructure. While this only supported a small subset of STIL language features, it handled everything we needed to continue our testing without delaying the timeline for the unexpected feature request. The extra fragility and reduced maintainability were okay, as the project only needed to last three more weeks. To this day, I still maintain it is the greatest single line of code I have ever written. Unfortunately, it was lost when the company was acquired, but it will live on forever in my heart.